The remaining basic atomic types are essentially all the same: they're all atomic integral types and have the same interface as each other, except that the associated built-in type is different. We'll look at them as a group.

5.2.5. Operations on standard atomic integral types

As well as the usual set of operations (`load()`, `store()`, `exchange()`, `compare_exchange_weak()`, and `compare_exchange_strong()`), the atomic integral types such as `std::atomic<int>` or `std::atomic<unsigned long long>` have quite a comprehensive set of operations available: `fetch_add()`, `fetch_sub()`, `fetch_and()`, `fetch_or()`, `fetch_xor()`, compound-assignment forms of these operations (`+=`, `-=`, `&=`, `|=`, and `^=`), and pre- and post-increment and decrement (`++x`, `x++`, `--x`, and `x--`). It's not quite the full set of compound-assignment operations you could do on a normal integral type, but it's close enough: only division, multiplication, and shift operators are missing. Because atomic integral values are typically used either as counters or as bitmasks, this isn't a particularly noticeable loss; additional operations can easily be done using `compare_exchange_weak()` in a loop, if required.

The semantics match closely to those of `fetch_add()` and `fetch_sub()` for `std::atomic<T*>`; the named functions atomically perform their operation and return the *old* value, whereas the compound-assignment operators return the *new* value. Pre- and post-increment and decrement work as usual: `++x` increments the variable and returns the new value, whereas `x++` increments the variable and returns the old value. As you'll be expecting by now, the result is a value of the associated integral type in both cases.

We've now looked at all the basic atomic types; all that remains is the generic `std::atomic<>` primary class template rather than the specializations, so let's look at that next.

5.2.6. The std::atomic<> primary class template

The presence of the primary template allows a user to create an atomic variant of a user-defined type, in addition to the standard atomic types. You can't use just any user-defined type with `std::atomic<>`, though; the type has to fulfill certain criteria. In order to use `std::atomic<UDT>` for some user-defined type `UDT`, this type must have a *trivial* copy-assignment operator. This means that the type must not have any virtual functions or virtual base classes and must use the compiler-generated copy-assignment operator. Not only that, but every base class and non-`static` data member of a user-defined type must also have a trivial copy-assignment operator. This essentially permits the compiler to use `memcpy()` or an equivalent operation for assignment operations, because there's no user-written code to run.

Finally, the type must be *bitwise equality comparable*. This goes alongside the assignment requirements; not only must you be able to copy an object of type `UDT` using `memcpy()`, but you must be able to compare instances for equality using `memcmp()`. This guarantee is required in order for compare/exchange operations to work.